Assignment 2: Theory & pthread

Bar-Ilan CS 89-3312
Parallel Systems Programming

November 9, 2026

1 Theory

In this assignment, you are required to practice parallel thinking. For each algorithmic solution, your answer must include the following components:

1. Pseudocode describing the algorithm.
2. A written explanation of how the algorithm works.
3. A formal proof of correctness of the algorithm.

   Note: The correctness proof must be mathematical and formal, similar in style to the proofs presented in the Algorithms 1 course. Handwritten or scanned explanations are **not allowed**.
4. A runtime analysis of the algorithm.

Handwritten, scanned, or photographed submissions will not be accepted. Solutions must be typed and submitted as PDF files.

1.1 Question1: parentheses

Determine whether a given mathematical equation containing parentheses has balanced parentheses or not.

Describe an algorithm that accepts the equation as input and returns `true` if the parentheses are balanced, and `false` otherwise.

Achieve a time complexity of $O(\log n)$.

Examples

- Input: `"(3 (4 + 2) - 1)"`
  Output: `true`
- Input: `"((5 2) / (4 - 2)"`
  Output: `false`

The equation may contain numbers, arithmetic operators (`+`, `-`, `*`, `/`), and parentheses. Balanced parentheses mean that every opening parenthesis has a corresponding closing parenthesis, and they are properly nested.

1.2 Question 2: Counting subarrays with a given sum

Given an array A of length n and a number b, we want to count the number of continuous subarrays whose sum is exactly b.

For example, given

$$A = [1, 7, 3, 5, 2, 6] \qquad \text{and} \qquad b = 8,$$

the output is 3, corresponding to the subarrays

$$[1, 7], \qquad [3, 5], \qquad [2, 6].$$

1. **Positive numbers only.**

   Assume that every element in the array is strictly positive, that is,

   $$A[i] > 0 \qquad \text{for every } i \in \{1, \ldots, n\}.$$

   Write an algorithm that counts the number of continuous subarrays whose sum is exactly b.

   Try to make your algorithm as efficient as possible.

2. **General case.**

   Now assume that the array may contain arbitrary numbers.

   In other words, each element can be positive, negative, or zero:

   $$A[i] \in \mathbb{Z} \qquad \text{for every } i \in \{1, \ldots, n\}.$$

   Write an algorithm that counts the number of continuous subarrays whose sum is exactly b.

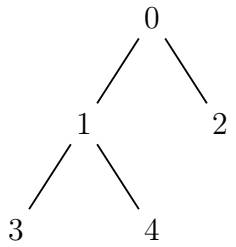
   Try to make your algorithm as efficient as possible.

1.3 Question3: k-th Ancestor in a Rooted Tree

Given a rooted tree with n nodes, each node has a unique identifier ranging from 0 to $n-1$. Each node i has a pointer to its parent node, and the root node points to itself.

Your task is to determine, for each node i, its k-th ancestor (the ancestor at distance k from the node). If no such ancestor exists, return -1 for that node.

Example



Parent pointers:

$$[0, 0, 0, 1, 1]$$

$$k = 2$$

Expected output:

$$[-1, -1, -1, 0, 0]$$

Achieve a parallel runtime of $O(\log(k) \cdot \log(n))$ for all cases.

Notice that the algorithm should work for general trees as well (not just binary).

1.4 Question 4: Trapping Rain Water

Given an array `height` of length n, where each element represents the height of a vertical bar, compute how much water can be trapped after raining.

The array `height` represents a map of bars, where the width of each bar is 1.

For example, for

$$\texttt{height} = [0, 2, 0, 3, 1, 0, 1, 3, 2, 1],$$

the total amount of trapped water is 9 units.

The PageRank Formula of a page v

$$Rank(v) = \frac{d}{N} + (1-d)\left(\sum_{u_i} \frac{Rank(u_i)}{outlink(u_i)} + \sum_{u_j} \frac{Rank(u_j)}{N}\right)$$

- d: Jump factor.
- N: Number of webpages
- u_i: Pages with links to v.
- u_j: Pages without outlinks.
- $Rank(u_i)$: The rank of the page u_i in the previous iteration.
- $outlinks(u_i)$: The number of pages u_i is pointing to.

Write an efficient algorithm that receives the array `height` and returns the total amount of trapped water.

Try to make your algorithm as efficient as possible.

2 Page Ranking Algorithm

PageRank (PR) is an algorithm used in real life by Google to rank web pages in their search results. In a nutshell, according to Google:

> "PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites."

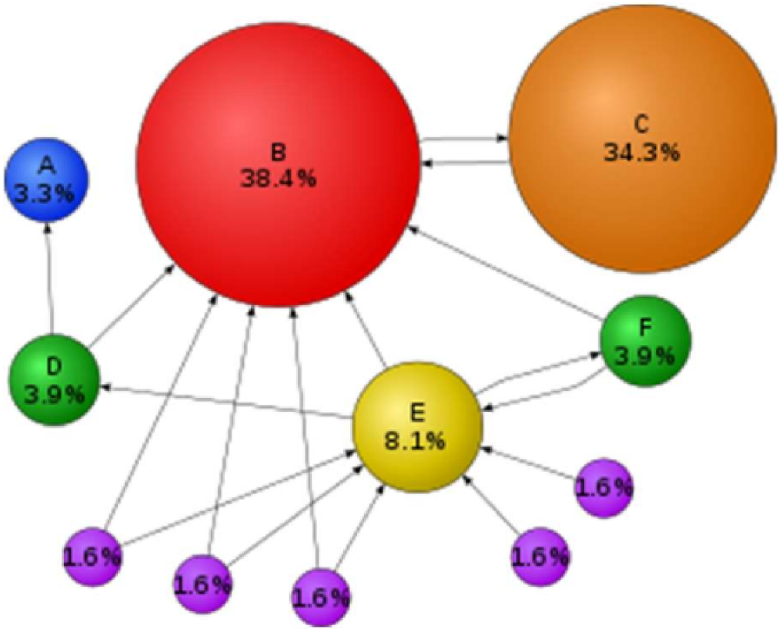
In simple words, PR algorithm gives each web page a score. For a vertex (webpage) A, we mark its score (page rank) as $PR(A)$. The page rank is used to understand the relative importance of each web page, compared to other web pages. Higher $PR \Rightarrow$ more relevant.

To be precise, note that Google uses additional algorithms, but this algorithm is still used and it is the first one which was used!

2.1 Notes

- Hyperlinks from a webpage to itself are ignored.
- If there are multiple links between two web pages, we still have a single edge between them.

Below is a simple illustration of the PR algorithm. The percentage show the perceived importance, and the edges represent hyperlinks, as explain above:



2.2 Motivation

In a perfect world, we could have just counted the number of links to a web page; however, as owners of an unpopular web page, **we could create millions of dummy pages linking to our page and fool the system.**

The formula uses a model of a "random surfer" who reaches their target site after several clicks, then switches to a random web page. In simple words, the PageRank score of a web page reflects the chance that the random surfer will land on that web page by clicking on a link.

2.3 Additional explanation (not required)

The PageRank theory holds that an imaginary surfer who is randomly clicking on links will eventually stop clicking. The probability, at any step, that the person will continue following links is a damping factor d. The probability that they instead jump to any random page is $1 - d$. It is generally assumed that d is around 0.85.

2.4 Pseudocode for the sequential algorithm

Here, `PageRank(G, n)` receives as input a graph G and n (the number of iterations). Notice that we initialize the rank of all pages to $\frac{1}{N}$, where N is the total number of webpages.

```
PageRank(G, n)
1 d <- 0.15
2 N <- number of nodes in G
3 For all nodes v in G:
4 Initialize Rank(v) = 1/N
5 While n > 0:
6 For all v with links from ui and uj with no outlinks:
7 Rank(v) = d/N + (1-d) * ( sum( Rank(ui)/outlink(ui) ) + sum( Rank(uj)/N ) )
8 n <- n - 1
9 return Rank
```

A nicer preview of the PageRank formula of a page v

The PageRank Formula of a page v

$$Rank(v) = \frac{d}{N} + (1-d)\left(\sum_{u_i} \frac{Rank(u_i)}{outlink(u_i)} + \sum_{u_j} \frac{Rank(u_j)}{N}\right)$$

- d: Jump factor.
- N: Number of webpages
- u_i: Pages with links to v.
- u_j: Pages without outlinks.
- $Rank(u_i)$: The rank of the page u_i in the previous iteration.
- $outlinks(u_i)$: The number of pages u_i is pointing to.

Explanation for tricky parts

- The first sigma: for all websites u_i that are pointing to v, we take their ranks and divide them by how many websites they are pointing to. This makes sense: if a page points to many other pages, the importance of each reference should decrease.
- The second sigma: relocating the probabilities for a vertex that doesn't have any outgoing links (edges). If we land on a website without outgoing links, we must quit and go to any other website at random. That is why the sum is divided by N (number of webpages), to uniformly redistribute probabilities.

2.5 Runtime

For a graph with N vertices and M directed edges, running n iterations gives runtime:

$$O\big(n \cdot (N + M)\big).$$

This is because in every iteration we traverse every vertex and all edges that come out of it.

2.6 Why parallelize?

There are nearly 2 billion websites online. Running a sequential algorithm on such a large graph is not practical. So we parallelize.

2.7 Your task (Parallel PageRank)

Your task is to create a parallel PageRank function which computes the `rank` array for each vertex in the graph.

Your API should contain only one function in the header file:

```
void PageRank(Graph* g, n , float* rank); // result will be returned to rank array.
```

2.8 Links with helpful information

- `https://en.wikipedia.org/wiki/PageRank`
- `https://churchill-aloha.medium.com/pagerank-algorithm-explanation-code-2fb6c0389bed`
- `https://www.youtube.com/watch?v=meonLcN7LD4`

3 Submission Guidelines

To reduce the likelihood of misunderstandings, please read and follow the instructions below carefully.

3.1 Implementation Requirements

You are provided with several files as part of the starter code:

- `graph.h`
- `graph.c` — contains functions for creating and managing the graph
- `thr_pool.c` — the thread pool implementation shown in class
- `thr_pool.h` — the corresponding header file shown in class
- `main.c` — a main program that constructs a graph and runs a serial PageRank algorithm, which you may use to compare against your parallel solution

You are required to implement the parallel PageRank algorithm according to the following rules:

- You must implement the PageRank function in `pagerank.c`.
- You must provide a corresponding header file `pagerank.h` containing the function declaration.
- **For the code part submit only** `pagerank.c` and `pagerank.h`.
- It is strongly recommended to use a thread pool.
- **Do not** use OpenMP. Only `pthreads` are allowed. Using OpenMP will result in an automatic grade of **0**.
- If you believe you have a more efficient graph representation that is better suited for this algorithm, discuss it with the teaching assistant in advance. The TA will decide whether it may be used.

3.2 Environment and Submission Format

1. The assignment may be completed individually or in pairs.
2. Submission is through the **submit system**.
3. Your PageRank implementation must compile and run without errors or warnings on BIU servers.
4. In the first line of every submitted source file, include a comment with your ID and full name. For example: `/* 123456789 Israela Israeli */`.
5. Failure to use parallel computation (i.e., `pthread` libraries for **PageRank**, and parallel algorithms for the **theory questions**) in any part of the assignment will result in an automatic grade of **0**. Sequential solutions are not acceptable.

6. All answers for the **Theory** part must be typed using a text editor such as LaTeX, LyX, or a word processor (e.g., Microsoft Word). Handwritten, scanned, or photographed submissions will **not** be accepted.

7. The theory answers must be submitted as a PDF file named `ass2.pdf`.

8. Submit a single zip file named `ass2.zip` containing:

   - `ass2.pdf`
   - `pagerank.c`
   - `pagerank.h`

9. **Deadline: 26.11.2026**